Name: Dau Vu Tuan Khoi

ID: 105709548

Technical Report Task 1: Setting Up the Environment and Analyzing the Stock Prediction Code v0.1

1. **Overview of Environment Setup**
   To deploy the stock prediction system v0.1 and the reference project P1, setting up a consistent computing environment is required. By analyzing the stock_prediction.py file, the system requires the following core libraries:
   - numpy: Handling multi-dimensional arrays and doing arithmetic calculations
   - matplotlib: A tool for data visualization and charts
   - pandas: Handles table-like data structures
   - tensorflow: A platform framework for building and training LSTM neural networks
   - scikit-learn: Provides preprocessing tools (like MinMaxScaler)
   - yfinance: The main library for downloading real financial data from Yahoo Finance
   - panda-datareader: Although imported in the source code, this is a legacy library and its role for fetching data has been replaced by yfinance in the current version.

   Contents of the requirements.txt file:

   - numpy
   - matplotlib
   - pandas
   - tensorflow
   - scikit-learn
   - pandas-datareader
   - yfinance

   The process of setting up a virtual environment

   Using a virtual environment is strongly recommended for two key technical reasons:

   1. **TensorFlow version sensitivity:** TensorFlow often has major changes between versions (breaking changes). Isolating it helps avoid conflicts with the system's global versions.

2. **Consistency for v0.1 and P1:** It ensures both projects run on a stable library platform, making benchmarking as accurate as possible.

2. System Test Run Results

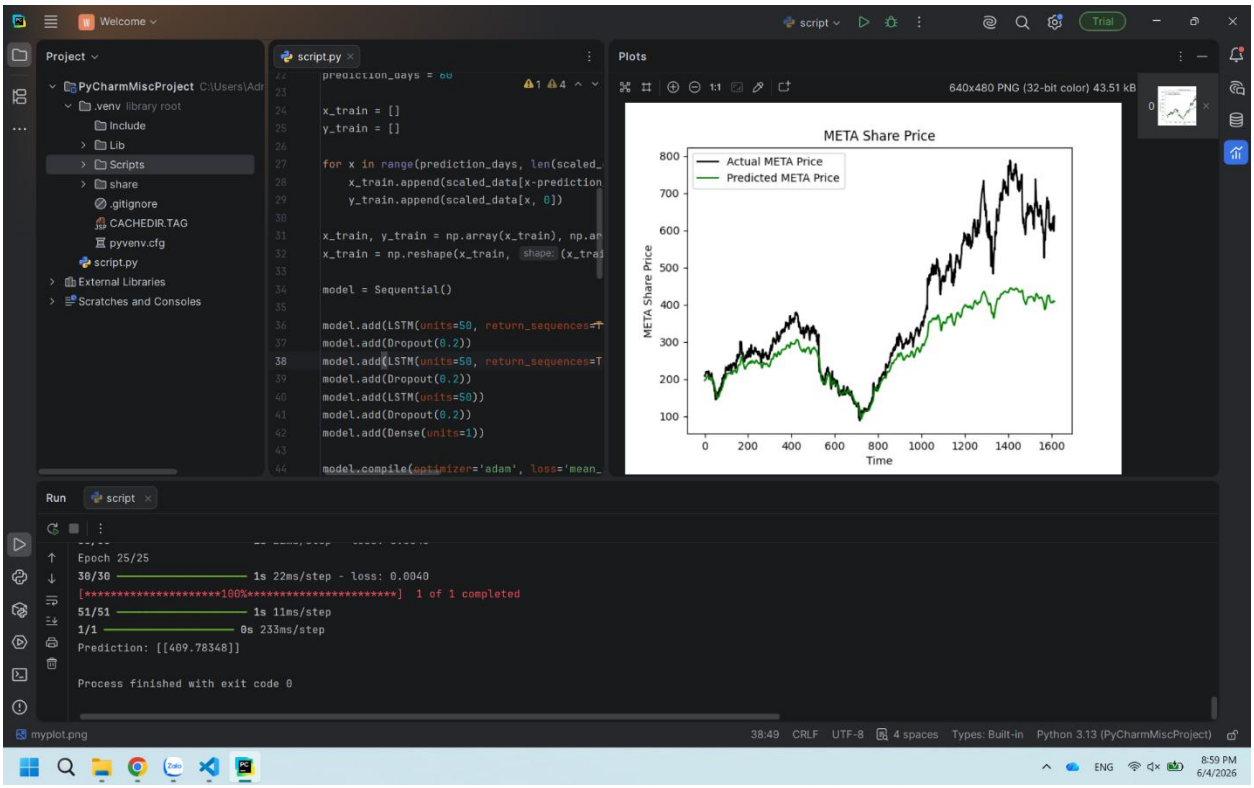
The testing process was carried out on the default stock code CBA.AX with specific time points based on the source code:

- **Training data:** 01/01/2020 to 01/08/2023
- **Test data:** 02/08/2023 to 02/07/2024

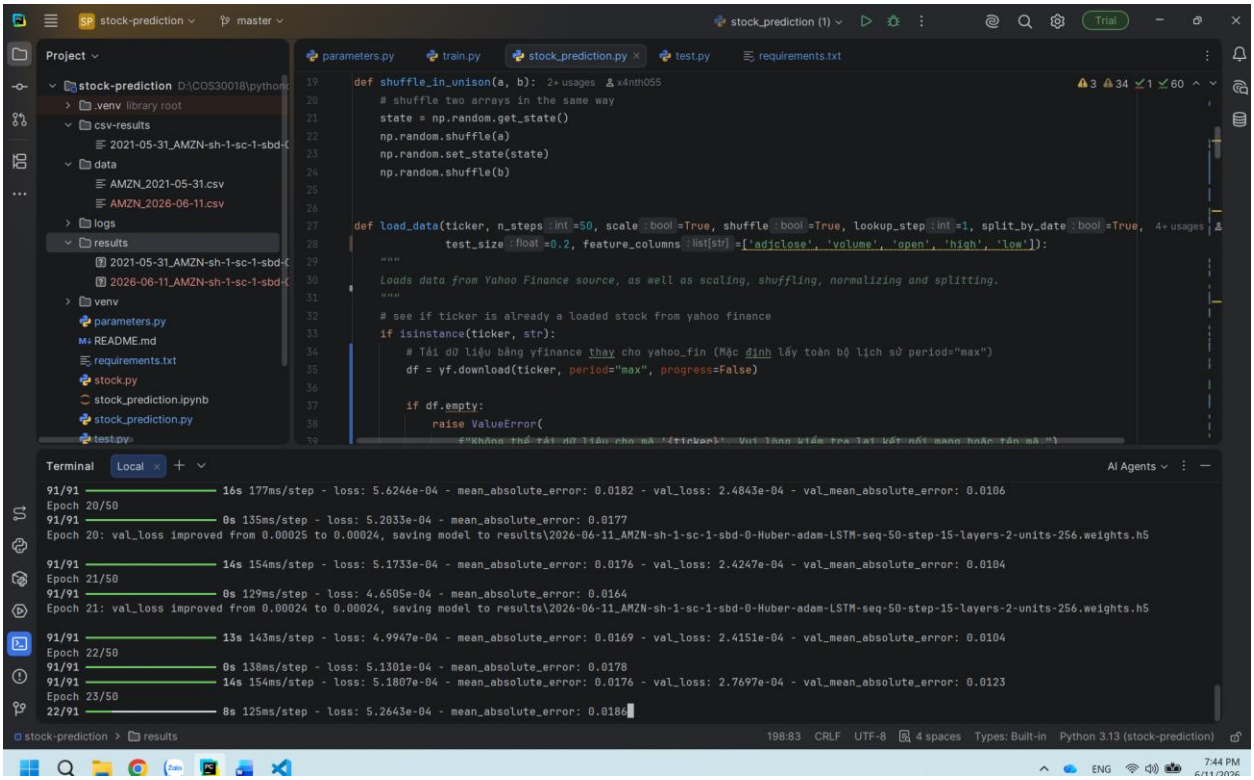
Standard execution process

The source code sequentially loads data via yfinance, normalizes it using MinMaxScaler, builds a 3-layer LSTM architecture, and trains it over 25 epochs. The final result is output as a chart comparing actual prices with predicted prices.

V0.1



P1



Execution status comparison Table

Project	Status	Response time	Forecast performance
v0.1	Success	2-5 minutes (Local CPU/GPU)	Poor (Error 10-13%)
P1	Success	Equivalent to v0.1	Significantly better

Note: Response time is based on empirical observation using internal hardware setup.

3. Detailed Analysis of Source Code v0.1 (Source Code Analysis)

Data Preprocessing Mechanism and Sliding Window Creation

The source code uses MinMaxScaler to bring the Close price data into the range (0, 1). An important parameter is PREDICTION_DAYS = 60, which acts as the size of the sliding window. The model will use data from 60 consecutive days to predict the price on the 61st day. Technically, the input data is transformed from a 2D array into a 3D tensor with the shape (p, q, 1), where:

- **p:** Number of data samples (total days – PREDICTION DAYS).
- **q:** Number of time steps, which is 60 days

- **1:** Number of features, here we only use the Close price

Stacked LSTM model architecture

The Sequential architecture of v0.1 includes the following layers:

1. **LSTM Layer 1 (50 units):** Declare input_shape (or you can use InputLayer as a cleaner alternative mentioned in code comments). Set return_sequences=True to pass the entire data sequence to the next layer.
2. **LSTM Layer 2 (50 units):** Keep return_sequences = True to continue building a Stacked structure.
3. **LSTM Layer 3 (50 units):** The last LSTM layer, outputs a single result for the Dense layer.
4. **Dropout (0.2):** Placed between layers to randomly drop 20% of neurons to help reduce overfitting.
5. **Dense (1 unit):** The final fully connected layer to give the actual predicted value.

Training setup

The system uses the Adam optimizer and the mean_squared_error (MSE) loss function. With epochs=25 and a batch_size of 32, the model updates weights based on the accumulated error from batches of 32 samples, helping balance between computation speed and convergence ability.

4. Performance Evaluation and Current Issues

Analysis of Errors and Model Failures

From an ML engineer's perspective, the current v0.1 model is a technically inadequate predictor. A next-day prediction error of 10-13% is way too high for financial trading.

ISSUE#2: The problem of standardization and the saturation zone.

The biggest issue lies in using a scaler that was 'fit' on the training data (2020-2023) for the test data (2023-2024)

- Mathematical consequence: If the stock price in the testing phase goes beyond the max/min range of the training phase, the transformed value will be outside the (0, 1) range.

- Model impact: These values push activation functions (like tanh or sigmoid) in the LSTM gates into saturation zones. Here, the gradient disappears, causing the model to make predictions that are completely off from reality.

5. **Comparing Benchmarking with Project P1**

   Definition of "Better Prediction"

   A model is considered "better" when it outperforms v0.1 in the following technical criteria:

   1. Mean Absolute Error (MAE): The average error between the predicted price and the actual price is the lowest.
   2. Directional Accuracy: The ability to correctly predict whether the price will go up or down the next dasy, rather than just matching numeric values.

   Analyze the differences

   Even though both v0.1 and P1 use Stacked LSTM, P1 gives more promising results. This comes from P1 applying better data processing techniques (Feature Engineering) and more professional training/test data splitting, avoiding data leakage or normalization errors like ISSUE #2 in v0.1.

   Key learning points from P1:

   - Advanced preprocessing: We need to find ways to handle data so that the model isn't affected by values outside the trained range.
   - Input diversification: Instead of just relying on the Close price, we should include other indicators to make it more stable.
   - Hyperparameter optimization: The number of units (50) and epochs (25) in v0.1 need to be returned for better performance.